

## Research Article

# A Configurable Accelerator for CNN-Based Remote Sensing Object Detection on FPGAs

Yingzhao Shao,<sup>1,2</sup> Jincheng Shang<sup>1,3</sup>, Yunsong Li,<sup>1</sup> Yueli Ding,<sup>2</sup> Mingming Zhang,<sup>2</sup> Ke Ren,<sup>2</sup> and Yang Liu<sup>2</sup>

<sup>1</sup>Xidian University, Xi'an, China

<sup>2</sup>China Academy of Space Technology (Xi'an), Xi'an, China

<sup>3</sup>Aeronautics Computing Technology Research Institute, Xi'an, China

Correspondence should be addressed to Jincheng Shang; shangjincheng@stu.xidian.edu.cn

Received 29 January 2024; Revised 3 June 2024; Accepted 4 June 2024

Academic Editor: Sudarshan K. Srinivasan

Copyright © 2024 Yingzhao Shao et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Convolutional neural networks (CNNs) have been widely used in satellite remote sensing. However, satellites in orbit with limited resources and power consumption cannot meet the storage and computing power requirements of current million-scale artificial intelligence models. This paper proposes a new generation of high flexibility and intelligent CNNs hardware accelerator for satellite remote sensing in order to make its computing carrier more lightweight and efficient. A data quantization scheme for INT16 or INT8 is designed based on the idea of dynamic fixed point numbers and is applied to different scenarios. The operation mode of the systolic array is divided into channel blocks, and the calculation method is optimized to increase the utilization of on-chip computing resources and enhance the calculation efficiency. An RTL-level CNNs field programmable gate arrays accelerator with microinstruction sequence scheduling data flow is then designed. The hardware framework is built upon the Xilinx VC709. The results show that, under INT16 or INT8 precision, the system achieves remarkable throughput in most convolutional layers of the network, with an average performance of 153.14 giga operations per second (GOPS) or 301.52 GOPS, which is close to the system's peak performance, taking full advantage of the platform's parallel computing capabilities.

## 1. Introduction

With the surge of computing power and data in recent years, neural network algorithms have made great progress and have shown better accuracy and efficiency than traditional algorithms in image, speech, video processing, and other tasks [1, 2, 3]. Breakthroughs in the architecture of various convolutional neural networks (CNNs) have greatly improved the accuracy of recognition and segmentation tasks. At the same time, these CNNs are becoming larger and more complex, requiring a large amount of computing resources to train and evaluate. However, it is still a challenging task to deploy a new generation of high-complexity, intelligent CNNs on orbiting satellites with limited computing resources.

Currently, three primary hardware platforms are utilized to accelerate CNNs: graphics processing units (GPUs) [2], application-specific integrated circuits (ASICs) [4, 5, 6], and field programmable gate arrays (FPGAs) [7, 8]. NVIDIA

actively promotes the utilization of GPUs for acceleration. In comparison to CPU architecture, GPUs feature more transistors within the same area, replacing cache and flow control, and are specifically designed for floating-point arithmetic, making them well-suited for deep learning data calculations. ASICs are customized AI acceleration chips tailored to the requirements of specific applications. They excel in particular scenarios and applications, boasting rapid processing speeds and low power consumption. However, they come with drawbacks such as high research and development costs, lengthy development cycles, and low replicability. FPGAs offer substantial computing power and flexibility. They represent semi-custom hardware that can be programmed to define cell configuration and link architecture for computing, resulting in high flexibility. FPGAs also deliver high performance per watt, making them appealing for large-scale server deployments or resource-constrained applications. Consequently, FPGAs are gaining popularity as an alternative solution.

The resources of in-orbit satellite payloads are very scarce. For example, China's Chang'e-2 lunar exploration satellite can only carry 166 kg payload (including 136 kg scientific payload and 30 kg engineering payload) with a launch weight of 2,480 kg, which is 6.69% [9]. At the same time, the inference operation of the CNN model itself is not sensitive to noise, and the weight range of each layer in the network is fixed with small relative fluctuation, so the weight data are suitable for low-bit quantization [10]. At present, the research on target detection network quantization is mainly focused on the inference side. This is because the quantization scheme cannot solve the problem of high-precision derivation during training, and the training is mainly based on the cloud environment, which has a high tolerance for power consumption. The inference side involves a large number of edge computing scenarios, and the on-board computing resources are limited. Low-bit data can reduce storage space occupancy and improve system power efficiency, so it is of practical significance to quantize on the inference side [11].

In conclusion, this paper introduces a hardware accelerator design for lightweight CNNs on FPGA. Our main contributions in this paper can be summarized as follows:

- (1) We propose a systolic array design with adaptable bandwidth that performs multiply-accumulate operations in convolution. The channel size of the input and output feature map of the array is tailored by configuring the expansion factor  $<T_i, T_o>$ .
- (2) We propose a lightweight architecture achieved through a dynamic fixed-point quantization scheme. The architecture currently implements INT16 and INT8 quantization, which can be selected according to resource and power requirements.
- (3) We propose a microinstruction-controlled accelerator utilizing a very long instruction word (VLIW) architecture. This design facilitates flexible configuration adjustments to accommodate diverse CNN computations.
- (4) We establish YOLOv3-Tiny and ResNet-18 network models through the configuration of predefined microinstruction. Evaluation of the accelerator performance of the network model is performed on the Xilinx VC709 FPGA chip. The experimental results show that our accelerator has great advantages in DSP utilization, performance, and power efficiency.

The remainder of the paper is structured as follows: In Section 2, we present the related works of satellite remote sensing, lightweight model, and FPGA accelerator. Section 3 outlines the accelerator design, including the accelerator architecture, the dynamic fixed-point quantization scheme, the systolic array design, the adapted convolution algorithm, and the microinstruction encoding. In Section 4, we analyze the experimental results of YOLOv3-Tiny and ResNet-18, including computations, throughput, comparison of throughput, DSP efficiency, and power efficiency with related work. Finally, in Section 5, we provide a conclusion to this work.

## 2. Related Work

**2.1. Research on Satellite Remote Sensing.** At present, there have been many advances in the application and research of onboard intelligent algorithms. The Jilin 1 [12] spectral satellite developed by China was launched in January 2019 with "one arrow and two satellites." The satellite carries multicore DSP pattern recognition technology, which has the function of automatically identifying, searching, and positioning forest fires and ships on the sea surface, effectively improving the efficiency of satellite data transmission and the timeliness of information acquisition. However, the parallelism of the DSP hardware platform is not superior to FPGA and ASIC, and it is difficult to accelerate large-scale parallel operations such as CNNs.

In addition to the technologies that have been applied on satellites, there are also some researches based on intelligent algorithms on satellites. Wei et al. [13] proposed a hybrid neural network model quantification method applied to remote sensing tasks. An acceleration structure of the seven-layer neural network is designed on Xilinx KC705 FPGA, and the classification accuracy on the MSTAR remote sensing dataset reaches 97.42%. However, the fixed hardware design makes it difficult to generalize to other models. Nong et al. [14] proposed a lightweight remote sensing target detection method based on embedded devices, which added a spatial attention module to the YOLOV3-Tiny network model to enhance the characteristics of remote sensing targets and evaluated the algorithm on the embedded platform Nvidia Jetson Xavier NX. The real-time detection speed can reach 32.5 FPS. This method effectively improves the detection accuracy of the YOLOV3-Tiny model and uses low-power devices to accelerate the algorithm. However, the acceleration efficiency of general-purpose embedded devices is obviously weaker than that of FPGA and ASIC, and there are few radiation-hardening methods for this kind of device. More experiments are needed to verify the stability of these devices in the space environment. Wang et al. [15] proposed a monolithic multiprocessing architecture for spaceborne SAR imaging and intelligent processing, and explored the pipeline execution method of SAR imaging algorithm and CNN algorithm. The architecture uses a strip-tiled data processing scheme to implement multitask model pipelining execution. In the Synopsys simulation platform, the chip based on this architecture can achieve 5.4 TOPS/W power efficiency when accelerating the VGG-11 network. This paper elaborates the design details of the deep pipeline but does not elaborate the design details of the encoding and execution of the operation instructions.

**2.2. Model Lightweight.** Since the introduction of AlexNet [2], CNNs have been evaluated primarily based on their performance and accuracy on ImageNet. This has led to the development of CNN architectures that prioritize these criteria without much consideration for model complexity and computational efficiency. For example, models such as AlexNet, VGGNet [16], GoogleNet, and ResNet [17] have evolved from eight layers to more than 100 layers. As the number of layers increases, so does the number of model

TABLE 1: Parameters and calculation amount of each model.

|                    | AlexNet | GoogLeNet | ResNet-50 | ResNet-18 | MobileNetv3 | YOLOv3-Tiny |
|--------------------|---------|-----------|-----------|-----------|-------------|-------------|
| Parameters ( $M$ ) | 60      | 7         | 25.6      | 11.2      | 5.4         | 8           |

parameters. Table 1 shows that an 18-layer residual network has more than 11.2 million parameters, resulting in a significant amount of storage space required for neural networks. In recent years, there has been a growing interest in lightweight CNNs [18]. Notable examples include MobileNetv1, MobileNetv2 [19], and MobileNetv3 [20], proposed by Google in 2017, 2018, and 2019, respectively, and YOLO9000 [21], YOLOv3 [22], and YOLOv4 [23] proposed in 2017, 2018, and 2020, respectively. In 2020, Adarsh et al. [24] explained a lightweight version of YOLOv3 called YOLOv3-Tiny, which still consists of over 8 million parameters.

Model size is not only a problem of memory capacity but also a problem of memory bandwidth. The model uses the weights in each inference. Image-related applications usually need to process data in real-time, and the resulting data transfer volume exceeds the memory bandwidth, making the calculation efficiency low. If CNN is to be deployed in special application environments such as satellites, lightweight models and high power efficiency are required to meet the memory power consumption limit of devices in unspecific scenarios.

The concept of lightweight AI aims to enhance the effectiveness of chips, platforms, and algorithms. While CNN traditionally employs data types like FP32 and FP64 for network training and inference, it is worth noting that INT16 and INT8 fixed-point numbers can sufficiently fulfill the requirements for network inference, offering an opportunity to optimize and compress the model [25, 26, 27].

In recent years, model lightweight and acceleration can be divided into two categories. From the perspective of network design, CNN model pruning is carried out, and from the perspective of quantization and compression, the weights, bias, and activations are quantized from FP32 floating-point values to INT16 or INT8 low-bit representations [28].

Regarding model pruning: In 2015, Chen et al. [29] introduced the Hashed Nets model, which incorporated a hash function and restructured the weights based on the Hamming distance between the parameters to achieve parameter sharing. In 2020, Xu et al. [30] presented a technique for network compression involving the multiplication of the number of convolution kernels in all convolutional layers by a scaling factor  $a$ , effectively reducing the computational workload by  $a$  times.

Quantization is also the main method of compression implementation of the CNN model [31]. In 2017, Google proposed an asymmetric quantization method [32], as shown in Figure 1. Both weights and activations are quantized to 8-bit integers, and a quantization training framework is added in the training phase to simulate the error caused by the quantization operation, which can minimize the accuracy loss caused by quantization in the real model. In 2019, Gong et al. [33] proposed the differentiable soft quantization (DSQ) method to construct a DSQ function, so that the

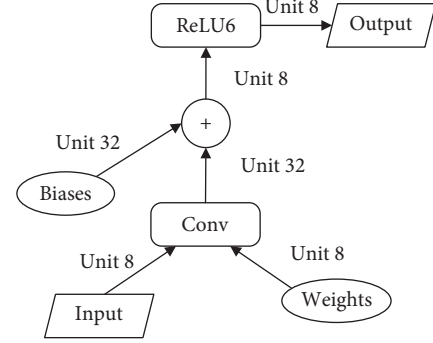


FIGURE 1: Convolutional 8-bit quantization process in deep learning.

quantization parameters can also be trained and learned along with the neural network. In 2020, Zhu et al. [34] introduced an error-sensitive learning rate adjustment method while quantizing network parameters to 8 bits and proposed a directional adaptive gradient stage processing method to deal with the accuracy loss caused by quantization.

**2.3. FPGA Accelerator.** During FPL 2016, Ma et al. [35] presented their research findings. Ma et al. [35] categorized network operations into modules such as the CONV module, POOL module, LRN module, FC module, etc., by dissecting the structure of network operation. This led to a reduction at the circuit level. Wei et al. [36] introduced a systolic array-based network accelerator design at DAC 2017. The design involves the array processing incoming features and weight data from both horizontal and vertical directions, with each operation unit being controlled by single instruction-stream multiple data-stream type instructions. This design eliminates multiplexers, conserves on-chip resources, and enables FPGA to operate at high frequencies. In the same year, at ISCA 2017, Shen et al. [37] proposed a flexible resource allocation scheme for object detection networks. They emphasized that the amount of computing data in each layer varies, and a fixed resource allocation mode leads to low utilization [37].

For board-level optimization, Zhang et al. [38] identified the optimal mapping method from each network layer to FPGA and introduced a multi-FPGA pipelined solution in 2016 to improve overall system performance. In 2019, Wang et al. [39] cascaded 4 VU9P server-grade chips to implement GoogLeNet network with 1.5 ms latency detection. Their FPL 2020 update incorporated the Winograd algorithm for sparsifying matrices, achieving a 2.9 times improvement [40].

### 3. Materials and Methods

**3.1. Accelerator Hardware Design.** The FPGA serves as the implementation platform of the system. After assessing the FPGA's hardware resources, including logic units, computational

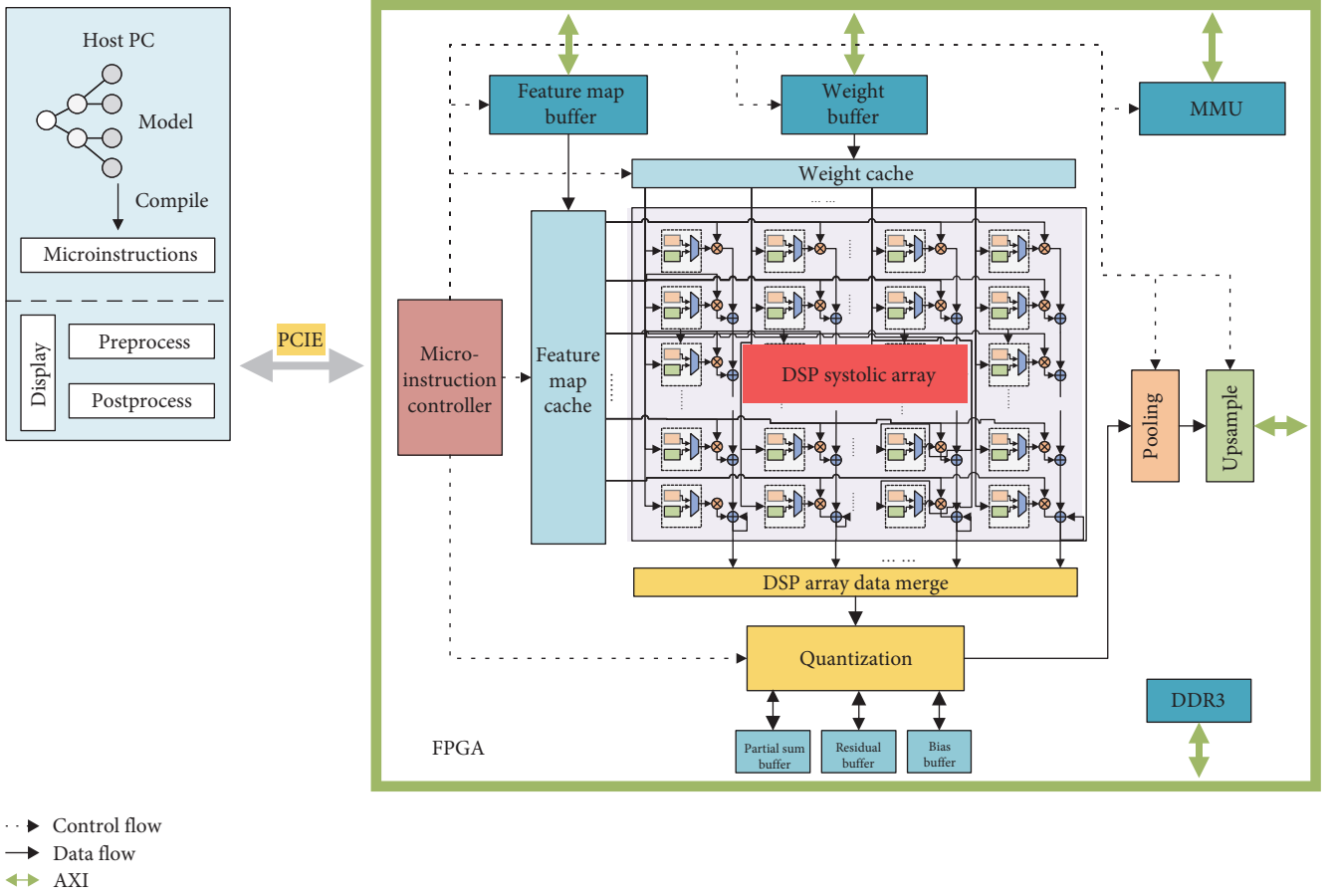


FIGURE 2: Accelerator architecture design.

capabilities, on-chip, and off-chip memory, the decision is made to employ the VLIW as the microinstruction encoding. This choice leverages the instruction-level parallelism inherent to VLIW, tapping into FPGA's high degree of parallelism to enhance hardware processing speed and resource utilization efficiency. Simultaneously, the utilization of instruction-driven hardware modules enhances the system's architectural flexibility. This adaptability enables the system to accommodate a wider range of neural network models by simply redesigning the microinstruction sequence, eliminating the need for a complete overhaul of the hardware structure and resulting in a more versatile architecture. The system also harnesses FPGA's DSP48 computing unit to accelerate the core convolution operation, thereby achieving efficient performance for the accelerator.

Figure 2 shows the architecture design of the accelerator, and the control core is the Microinstruction controller module to issue the entire system microinstruction stream. The memory management unit carries out DMA operation and transfers the off-chip DDR data to the on-chip. The DSP systolic array is the computing core, which accelerates the convolution operation and receives data from the feature map cache and the weight cache module. The quantization module consists of a series of operations, including quantization order, partial sum accumulation, linear activation, and residual operation.

**3.2. Dynamic Fixed-Point Quantization.** Based on the idea of dynamic fixed-point numbers, this article designs a data quantization scheme for INT8 or INT16, which is applied to different detection scenarios. The core idea of the dynamic fixed-point number method is to group the data in the network, so that the data in the same group share the same order, to hide the data order information, save on-chip storage, and simplify the hardware quantization complexity. The dynamic fixed-point number calculation equations are shown in Equations (1) and (2), respectively.

$$V = (-1)^S \times M, \quad (1)$$

$$M = 2^\beta \sum_{i=0}^{\gamma} 2^i F_i, \quad i \in [0, \gamma - 2], \quad (2)$$

where  $\beta$  is the order code parameter and  $\gamma$  is the fixed-point bit width.

In contrast to single-precision floating-point numbers, dynamic fixed-point numbers offer greater flexibility in representing data of varying magnitudes. For example, a 16-bit fixed-point number with  $\beta = 4$  can encompass the range of  $[-2, 048.9375, 2, 048.9375]$ . The dynamic fixed-point number approach proves advantageous for enhancing synergy between network accelerators, particularly in FPGA-based

applications. Since FPGA platforms allow for complete customization and support data operations of arbitrary bit width, the dynamic fixed-point number method can adapt to the specific requirements of an application scenario, effectively mapping network data onto the hardware side to facilitate FPGA deployment. Furthermore, different layers within the network may have varying data ranges for the feature map data and weights. Leveraging the dynamic fixed-point number scheme, it becomes possible to flexibly design different quantization strategies for data types and layers. This enables customized solutions, such as low-precision with a broad range or high-precision with a limited range, to maximize the utilization of quantized data based on distinct usage scenarios. In light of practical applications, the quantization techniques discussed are primarily geared toward inference. More precisely, the dynamic fixed-point number method replaces floating-point data such as FP64 and FP32 in the network with fixed-point integer data INT16 and INT8, offering advantages in both computation and storage.

**3.3. DSP Systolic Array.** The core of convolution neural network acceleration is the acceleration of convolution multiplication and accumulation operation. DSP48 is a kind of computing resource in FPGA that supports multiplication, accumulation, addition, barrel shift, and other functions. It is a proprietary computing core designed by Xilinx in its FPGA chip products. The DSP48E1 firmware is deployed on the chip of the VC709 development board. It has a high working frequency, is suitable for integer processing, multiplication, and addition operations, and is conducive to the acceleration of CNN convolution operations. Multiple DSP units are cascaded, and DSP with different working modes are used to form a multiply accumulation chain, so as to realize parallel pipeline multiply-accumulation operations.

Because the convolution operation of each CNN layer has strict data dependence and works serially between layers, the hardware structure design adopts the interlayer serial and intralayer parallel computing structure. According to memory bandwidth and resource utilization, the intralayer computing and processing resources are reasonably tiled, and the pipeline is fully used to improve the efficiency of on-chip resource utilization and improve system throughput. Therefore, this paper proposes a  $m \times n$  DSP systolic array that can be flexible with bandwidth to match the memory bandwidth for the acceleration of CNN operation. When ping-pong operations are introduced in the column direction,  $m \times 2n$  parallel computations can be performed simultaneously. The DSP array can perform multichannel parallel input of feature maps and output multichannel parallel output of feature maps. The operations in the direction of the input feature map channel are tiled to form a multiply-accumulation chain in the vertical direction of the array, and the output channel operations are calculated in parallel by the lateral DSP to improve the utilization of on-chip computing resources. At the same time, the calculation mode is optimized to improve the efficiency of computing resource utilization. Figure 3 shows a  $32 \times 16$  DSP array deployed on the VC709 platform.

As shown in Figures 4 and 5, the step size is  $S$ , the input feature map channel is  $C_{in}$ , and the output feature map channel is  $C_{out}$ , when the ordinary convolution is used for convolution operation, the  $K \times K$  multiplication and accumulation operation of the convolution kernel in the channel is priority, and the channel orientation feature map data accumulation is followed. The overall design idea of the DSP array is to expand the operation loop in Figure 5(a) according to the operation in Figure 5(b), as shown in Figure 6, the block convolution operation mode.  $\langle T_i, T_o \rangle$  is the expansion factor of the feature map, with  $T_i$  as the channel expansion factor of the input feature map,  $T_o$  as the channel expansion factor of the output feature map. The inner loop number  $Iter_o$  and the outer loop number  $Iter_i$  are shown in Equations (3) and (4), respectively. The cyclic block convolution operation is as in Equation (5), and a single block only performs the convolution operation between the  $T_i$  dimension of the input feature and the  $T_i$  dimension of  $T_o$  convolution kernels.

$$Iter_o = \frac{C_{out}}{T_o}, \quad (3)$$

$$Iter_i = \frac{C_{in}}{T_i}, \quad (4)$$

$$F_{out} = \sum_{\text{Slice}} \sum_{\text{Split}}^{Iter_i \ Iter_o} \text{Conv}(F_{in}(\text{slice} \times T_i), W(\text{split} \times T_o, \text{slice} \times t_i)), \quad (5)$$

where  $F_{in}$  and  $F_{out}$  indicate the input and output feature maps, respectively.  $W$  indicates the weights. Slice and split represent the input and output feature map channel orientation blocks, respectively.

Considering the high parallel computing characteristics of FPGA, we need to parallelize and pipeline the traditional software algorithm when designing the accelerator. In Figure 5, although traditional convolution looks much simpler, it will consume a lot of resources when it is directly deployed on the hardware, because the traditional convolution algorithm is a sequential process and needs to cache the whole image and all weights for each calculation. However, the convolution calculation method we proposed is a cyclic block convolution with  $T_i$  and  $T_o$  as the basic units in the input channel and output channel, respectively. Here, the loop process is pipelined, and the single block convolution calculation is parallel.

A  $32 \times 16$  array is selected as  $\langle T_i, T_o \rangle = \langle 32, 32 \rangle$ , and the feature map channel is divided into blocks. This is because the bit width of the AXI bus on VC709 is 512 bit, each pixel and weight of the feature map occupies 16-bit memory after INT16 quantization, and the pixel value or weight of the feature map of 32 channels at the same pixel position can be transmitted by 512-bit width. In the system design, the feature maps of most convolutional layers are cashed on-chip for operation, which is not limited by the

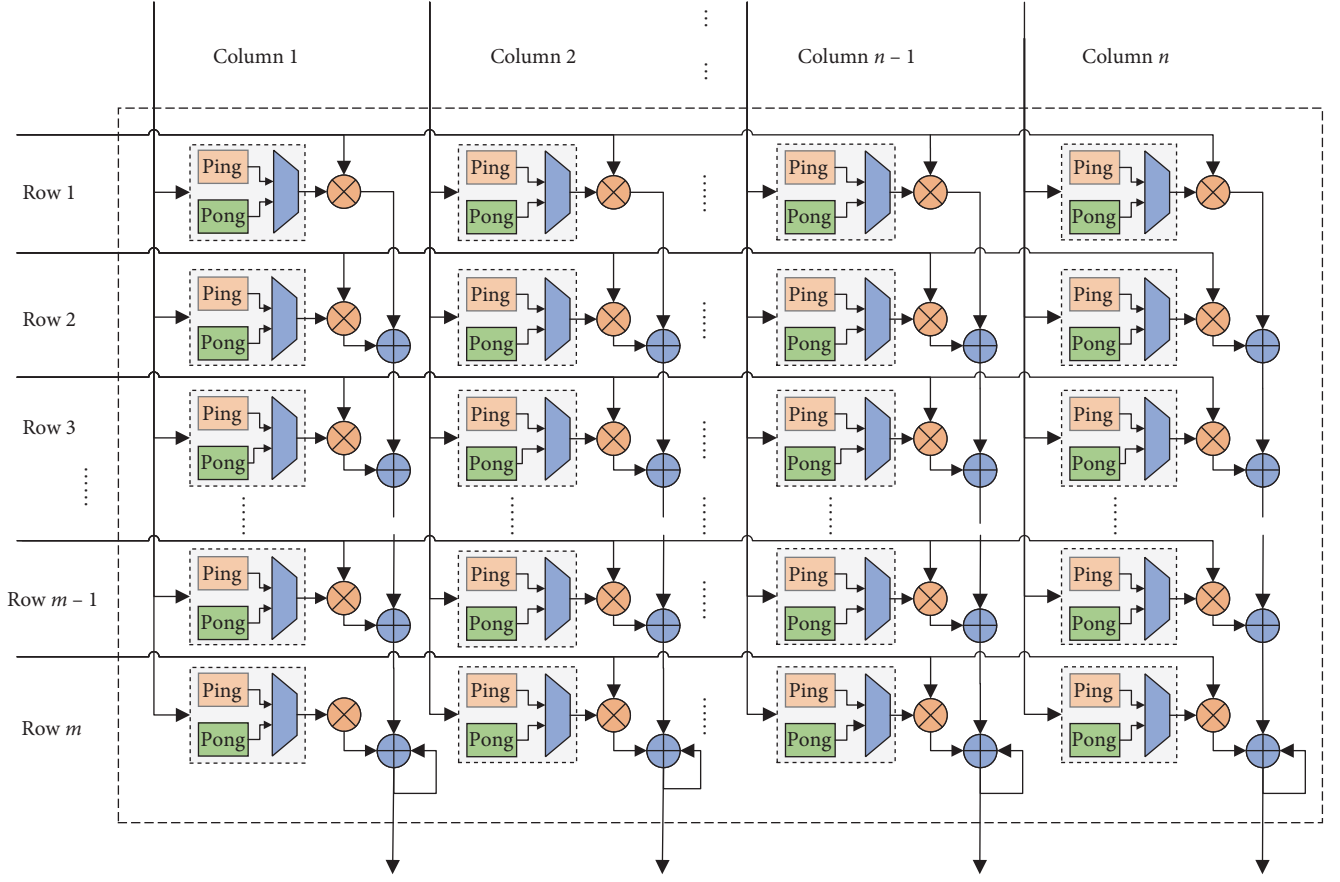


FIGURE 3: DSP array design.

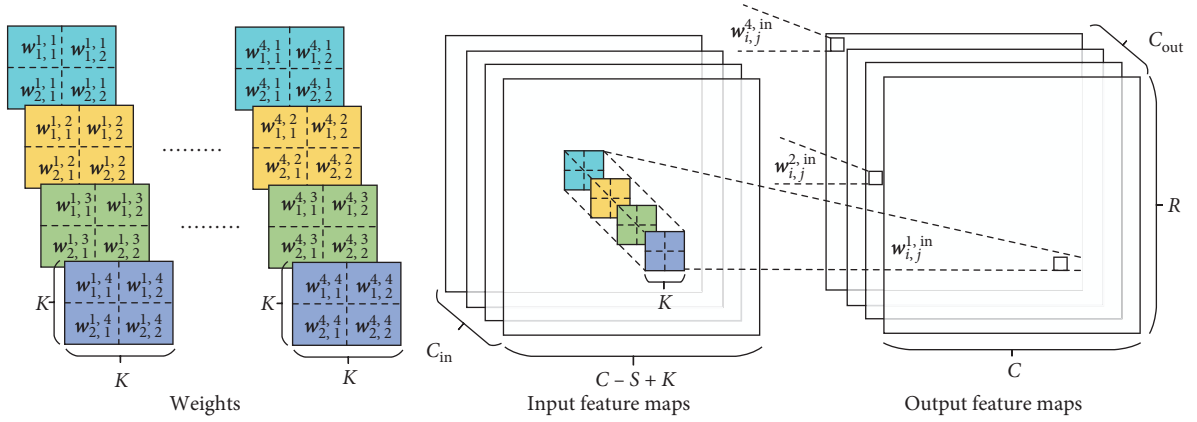


FIGURE 4: Ordinary convolution operation.

memory bandwidth. The factor  $T_i$  is fixed at 32, and  $T_o$  is selected as an integer multiple of 32 for channel expansion according to the bit width of the system, that is,  $\langle T_i, T_o \rangle = \langle 32, 32 \times \text{NUM} \rangle$ .

Under INT16 quantization,  $\text{NUM} = 1$ , the input feature map is a 32-channel block, the output feature map is a 32-channel operation cycle unrolled, and a  $32 \times 16$  DSP systolic array is designed for bandwidth utilization test. To adapt to bandwidth expansion, the  $32 \times 16$  array is used as the base unit for column expansion to increase the expansion factor

of the output channel. Each column of the array is multiplied and accumulated for the direction operation of the input feature map channel, and all DSPs in the lateral direction share the pixel value of the feature map of the same channel to carry out the parallel calculation of the output feature map channel direction. The 16-column DSP performed the ping-pong operation to perform the convolution calculation of the 32 output channels. By dividing the feature map into 32 channels and accumulating channel direction data preferentially, the data processing ability of the DSP array is

```

for ( row = 0; row < R; row ++ ) {
  for ( col = 0; col < C; col ++ ) {
    for ( out = 0; out < Cout; out ++ ) {
      for ( in = 0; in < Cin; in ++ ) {
        for ( i = 0; i < K; i ++ ) {
          for ( j = 0; j < K; j ++ ) {
            Loop: fmap_out [ out ] [ row ] [ col ] +=
                  weights [ out ] [ in ] [ i ] [ j ] *
                  fmap_in [ in ] [ S * row + i ] [ S * col + j ];
          } } } } } }
} } } } }

```

(a)

```

for ( slice = 0; slice < ( Cin / Ti ); slice ++ ) {
  for ( row = 0; row < R; row ++ ) {
    for ( col = 0; col < C; col ++ ) {
      for ( split = 0; split < ( Cout / To ); split ++ ) {
        for ( co = 0; co < To; co ++ ) {
          for ( i = 0; i < K; i ++ ) {
            for ( j = 0; j < K; j ++ ) {
              for ( ci = 0; ci < Ti; ci ++ ) {
                Loop: fmap_out [ split * To + co ] [ row ] [ col ] +=
                      weights [ split * To + co ] [ slice * Ti + ci ] [ i ] [ j ] *
                      fmap_in [ slice * Ti + ci ] [ S * row + i ] [ S * col + j ];
              } } } } } } } } } } } }
} } } } }

```

(b)

FIGURE 5: Convolution operation: (a) ordinary convolution operation; (b) DSP systolic array convolution operation.

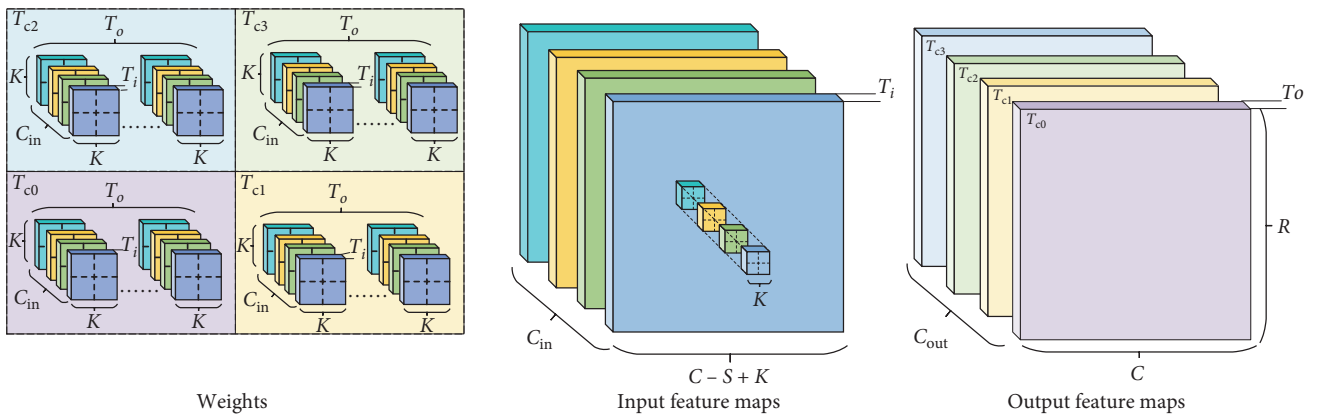


FIGURE 6: DSP systolic array convolution operation.

improved, and the throughput of the system is improved. By setting  $NUM = 2$ , we can produce two systolic arrays when using INT8 quantization with a 512-bit AXI bus width.

The operation mode of the first row of DSP is two 16-bit number operations of  $A \times B$ , the middle row is multiplied and accumulated in  $A \times B + PCIN$  mode, and the convolution operation greater than  $1 \times 1$  involves the multiplication and accumulation of similar pixels in the same channel. Therefore, the corresponding numerical accumulation logic is added after the last row of the DSP operation to accumulate the data in the channel. So, the actual mode of operation of the last line is  $A \times B + PCIN + C$ . Compared to the DSP design for convolution kernel acceleration, the proposed array can ensure high DSP resource utilization when performing convolution operations with convolution kernels of different sizes.

According to this hardware design, the theoretical peak throughput (TPT) can be calculated by Equation (6).

$$TPT = NUM \times nCols \times nRows \times nMACs \times f, \quad (6)$$

where  $NUM$  indicates the number of systolic array cores.  $nCols$  and  $nRows$  indicate the number of columns and rows of the DSP array, respectively.  $nMACs$  represents the number of multiplications and accumulations of a single

DSP block.  $f$  represents the frequency of the DSP array. In this paper, the DSP array works at 200 MHz, and the size is  $32 \times 16$ . So, the theoretical peak throughput of the INT16 quantization system can reach 204.8 giga operations per second (GOPS), while the INT8 quantization system can reach 409.6 GOPS.

**3.4. Microinstruction Control.** Microinstruction serves to manage the storage and operation modules, ensuring the system's flexibility and adaptability. In the accelerator, the microinstruction module mainly performs the fetch and decode functions of VLIW and the microinstruction distribution. These two tasks correspond to two sub-modules in the design, which are used to decode and distribute. After the instructions for the network structure are designed, the VLIW is pregenerated by the software compiler and stored in the on-chip RAM. After the system is powered on, the system executes the microinstruction sequence in the memory in order to realize the network computing function.

By analyzing the operation parameters and data characteristics of each layer of the quantized algorithm model and combining the address space division and the memory space size of the hardware memory system, the microinstruction parameters are determined. When data is stored on the chip, the gap between two sets of data is reduced in the address arrangement to reduce the use of on-chip resources. Each

TABLE 2: Microinstruction code.

| Type          | Name     | Address | Length (bits) |
|---------------|----------|---------|---------------|
| Memory access | rrmovf   | 0x000   | 109           |
|               | rrmovf   | 0x04a   | 74            |
|               | rrmovw   | 0x094   | 109           |
|               | rrmovw   | 0x101   | 74            |
|               | rimovb   | 0x14f   | 9             |
|               | immovf   | 0x158   | 73            |
| Operation     | psum     | 0x14b   | 2             |
|               | residual | 0x14d   | 2             |
|               | quant    | 0x1a1   | 13            |
|               | relu     | 0x1de   | 1             |
|               | pool     | 0x1ab   | 3             |
|               | upsample | 0x1e5   | 1             |
|               | pad      | 0x1af   | 6             |
|               | conv     | 0x1ca   | 6             |
| Control       | end      | 0x1fd   | 1             |

VLIW includes memory access microinstruction, operation microinstruction, and control microinstruction. Some microinstruction words are shown in Table 2. At the same time, the microinstruction design should reserve the expansion field to facilitate the access of new control signals after the system accesses the new module and improve the flexibility of the system.

The compiler uses a convolutional block as the core and writes different types of other layer operation parameters into a microinstruction; that is, a microinstruction can drive the data flow to complete a convolutional block and other types of operator operations required. The number of instructions in the microinstruction sequence is also determined by the number of blocks (multiple microinstruction representations are generated when the size of the block data is larger than the on-chip cache). The number of microinstructions required for single-layer convolution is shown in Equation (7).

$$\text{Inst\_num} = \text{Itero} \times \text{Iteri}. \quad (7)$$

## 4. Experiments

In this paper, the RTL level accelerator is implemented by Verilog HDL, and the single-core (INT16 quantization) and dual-core (INT8 quantization) experiments are carried out on Xilinx VC709 FPGA. The EDA tool is Vivado 2018.3, and the simulation tool is ModelSim 10.6d. The host PC with CPU: Intel Core i7-7700@3.60 GHz; Operating system: Ubuntu16.04; PCIe driver: Xilinx XDMA is used. We select YOLOv3-Tiny, a 24-layer lightweight object detection algorithm model, as the experimental model, and the experimental process is shown in Figure 7.

**4.1. Resource Utilization.** The resource utilization of the accelerator proposed in this paper on the Xilinx VC709 platform is shown in Table 3. The first row in Table 3 shows the

resource consumption of the microcontroller unit and microinstruction register (ROM). It can be seen that the microinstruction control logic in this paper consumes less on-chip resources, and the main resource consumption is less than 3%. The second row represents the resource consumption when INT16 quantization is adopted to generate a single DSP systolic array core. While the third row represents the dual DSP systolic array core generated under INT8 quantization.

**4.2. Accuracy Evaluation of Accelerator.** This section evaluates the accuracy of the quantization approach used in this study for object detection on the NWPU VHR-10 remote sensing dataset. The dataset contains a total of 800 remote sensing images with 10 classes of detection objects, and the test input image size is  $256 \times 256$ . The detection effect is shown in Figure 8. In addition to the model experiments of YOLOv3-Tiny, this paper also conducts the evaluation of the ResNet-18 model. The accelerator architecture proposed in this paper covers different kernel sizes ( $1 \times 1$ ,  $3 \times 3$ ), strides (1, 2), and layer types (conventional-CONV, DW-CONV). To accommodate the  $7 \times 7$  convolutions in the first layer of the original ResNet-18, we adapt them into three  $3 \times 3$  convolutions. The resulting structure of the adjusted ResNet-18, consisting of a 23-layer backbone network, is compiled using microinstruction, producing a total of 1,371 microinstructions, while YOLOv3-Tiny uses a total of 1,403 microinstructions. We use a dynamic fixed-point quantization scheme to quantize YOLOv3-Tiny and ResNet-18 to 16-bit and 8-bit, respectively, and present accuracy (IOU = 0.5) in Table 4.

According to the accuracy comparison, both quantization models show lossless performance when the common IoU setting is 0.5, and the quantization strategy can effectively reduce the usage of storage resources, as shown in Table 5.

**4.3. Performance Evaluation of Accelerator.** The amounts of operation of each layer in the network are shown in Figure 9. Where Figure 9(a) is the computation of the 13-layer convolution operator YOLOv3-Tiny, that is, the number of multiplication or addition operations, and the unit is giga operations (GOPs). The network incorporates only two types of convolution: CONV8, CONV10, CONV11, and CONV13 are  $1 \times 1$  convolutions, while the rest are  $3 \times 3$  convolutions. The  $3 \times 3$  convolution operation accounts for 91.3% of the multiplication and accumulation calculation of the model, which is the core operation of the model reasoning. The 23-layer backbone network ResNet-18 is shown in Figure 9(b). Where CONV1, CONV4, CONV9, CONV14, and CONV19 are  $1 \times 1$  convolutions, while the rest are  $3 \times 3$  convolutions.

According to the amount of computation in each layer, the throughput in each layer of the network can be calculated by Equation (8).

$$\text{Throughput} = \frac{\text{Computation}}{T}, \quad (8)$$

where  $T$  is the computation delay at each layer of the network. As shown in Figure 10, Figure 10(a) shows the

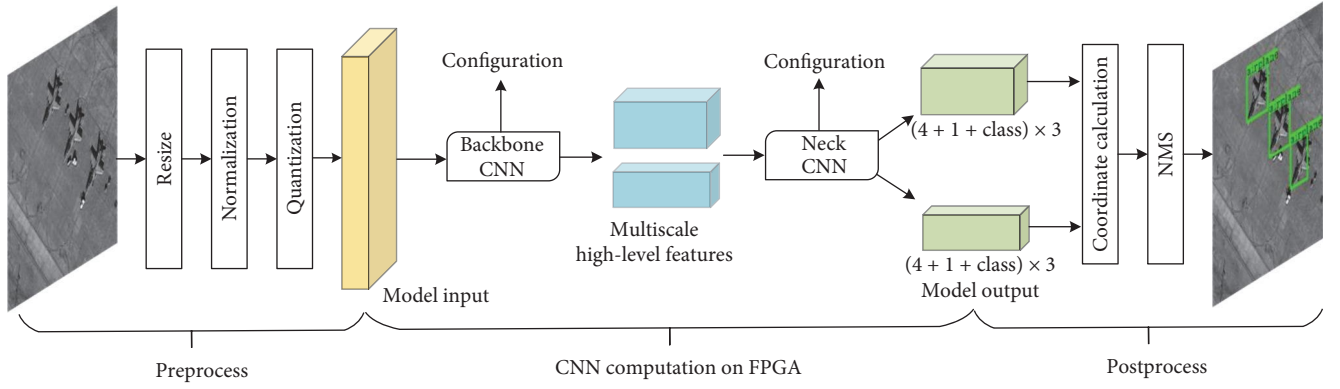


FIGURE 7: Flow of remote sensing object detection.

TABLE 3: FPGA resource utilization.

| Component        | LUT              | FF               | BRAM           | DSP            |
|------------------|------------------|------------------|----------------|----------------|
| Microinstruction | 2,653 (0.06%)    | 1,312 (0.15%)    | 32 (2.18%)     | 0 (0.00%)      |
| Single-core      | 168,930 (39.00%) | 139,490 (16.10%) | 483.5 (32.89%) | 514 (14.28%)   |
| Dual-core        | 224,088 (51.73%) | 173,281 (20.00%) | 831 (56.53%)   | 1,028 (28.56%) |

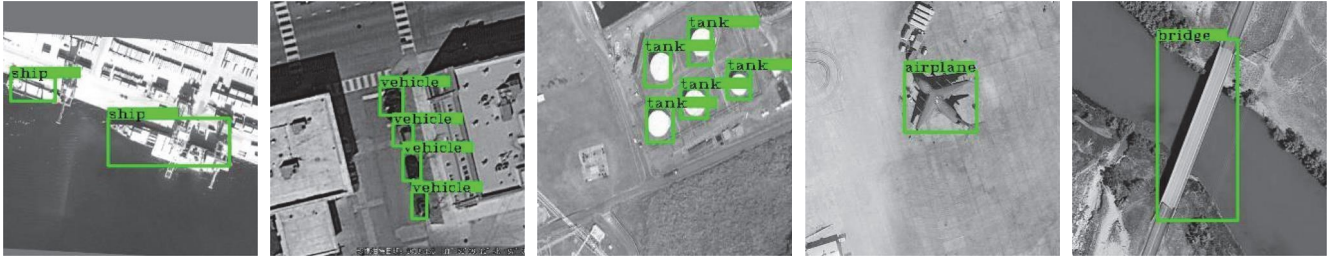


FIGURE 8: Effect of remote sensing object detection.

TABLE 4: Accuracy evaluated on NWPU VHR-10 dataset.

| Network     | 32-bit float | 16-bit fixed | 8-bit fixed |
|-------------|--------------|--------------|-------------|
| YOLOv3-Tiny | 0.959        | 0.959        | 0.96        |
| ResNet-18   | 0.810        | 0.810        | 0.845       |

TABLE 5: Size of the model after quantization.

|                         | 32-bit float | 16-bit fixed | 8-bit fixed |
|-------------------------|--------------|--------------|-------------|
| Model size (MB)         | 34.8         | 16.8         | 8.5         |
| Single-layer parameters | 18.0 MB      | 18.0 KB      | 9.0 KB      |

performance of YOLOv3-Tiny dual-core under INT8 quantization. In general, the  $3 \times 3$  convolutions with large amount of computation have high throughput, and the peak throughput reaches 378.6 GOPS, which is very close to the theoretical peak throughput of computing resources, 409.6 GOPS of computing resources (DSP usage: 1,028). The throughput of  $3 \times 3$  and  $1 \times 1$  convolution operations is very different. The reason is that the  $1 \times 1$  convolution has a lower data reuse rate for the feature map, which requires frequent

access to the feature map, resulting in the interruption of the DSP array pipeline and reduction of throughput, and reuse is also lower when the step size is 2. Figure 10(b) shows the performance of ResNet-18 single-core under INT16 quantization. The execution performance of instructions is almost the same as that of the YOLOv3-Tiny model, and it has a high throughput on  $3 \times 3$  convolution. The peak throughput reaches 199.8 GOPS, which is very close to the theoretical peak throughput, 204.8 GOPS of computing resources (DSP usage: 514).

Through the analysis, it can be seen that using the instruction-driven accelerator to optimize the scheduling of the system makes the throughput of the system in the most convolutional layers of the network close to the system peak, giving full play to the advantages of platform parallel computing. However, throughput also varies between  $3 \times 3$  convolutions. The reason is that in some layers with low computation but deep channels, the decoding microinstruction sequences are too long, resulting in performance loss.

**4.4. Comparison with CPU and GPU.** The spaceborne system is very sensitive to power consumption, so this section compares the performance of the FPGA accelerator proposed in

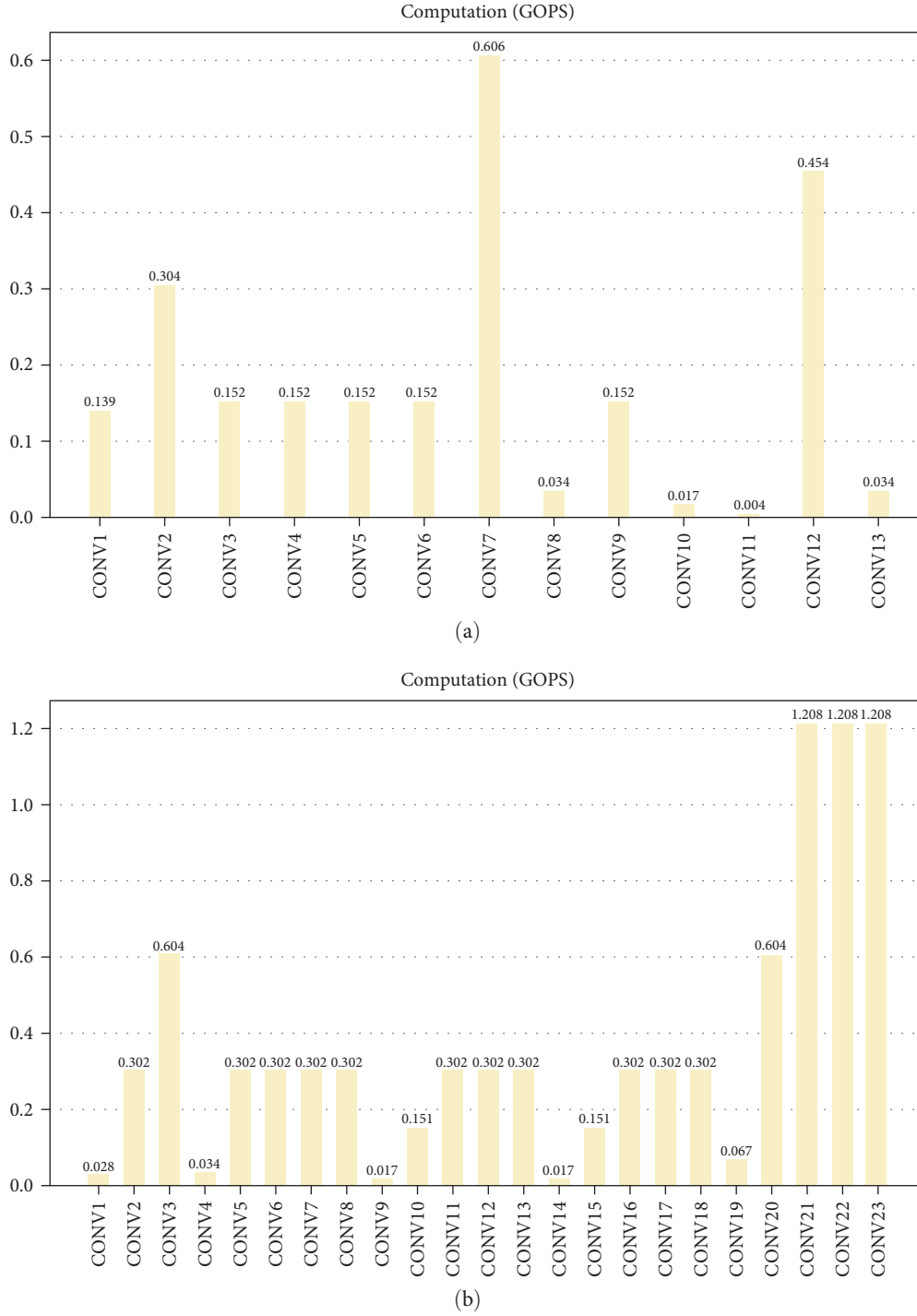


FIGURE 9: The amounts of operation at each layer of the network: (a) YOLOv3-Tiny; (b) ResNet-18.

this paper with other general-purpose hardware platform acceleration methods in terms of power efficiency. In this section, the speedup power efficiency of the same CNN model (YOLOv3-Tiny) on different platforms is tested, and the results are shown in Table 6. The test platforms are Intel Core i7-7700@3.60GHz CPU, NVIDIA GTX1080 GPU, and Xilinx VC709 FPGA. The software framework is DarkNet, and it takes 67s to test 266 remote sensing dataset images when the compilation option does not select GPU and 3s

when the GPU is turned on. Therefore, the CPU and GPU frame rates are 3.9 FPS and 88.7 FPS respectively. However, the frame rates of the proposed accelerator are 51 and 102 FPS, respectively. Compared with GPU, this paper has about 15 (single-core) and 19.4 (dual-core) times improvement in power efficiency.

**4.5. Comparison with Related Work.** We establish YOLOv3-Tiny and ResNet-18 network models through the configuration

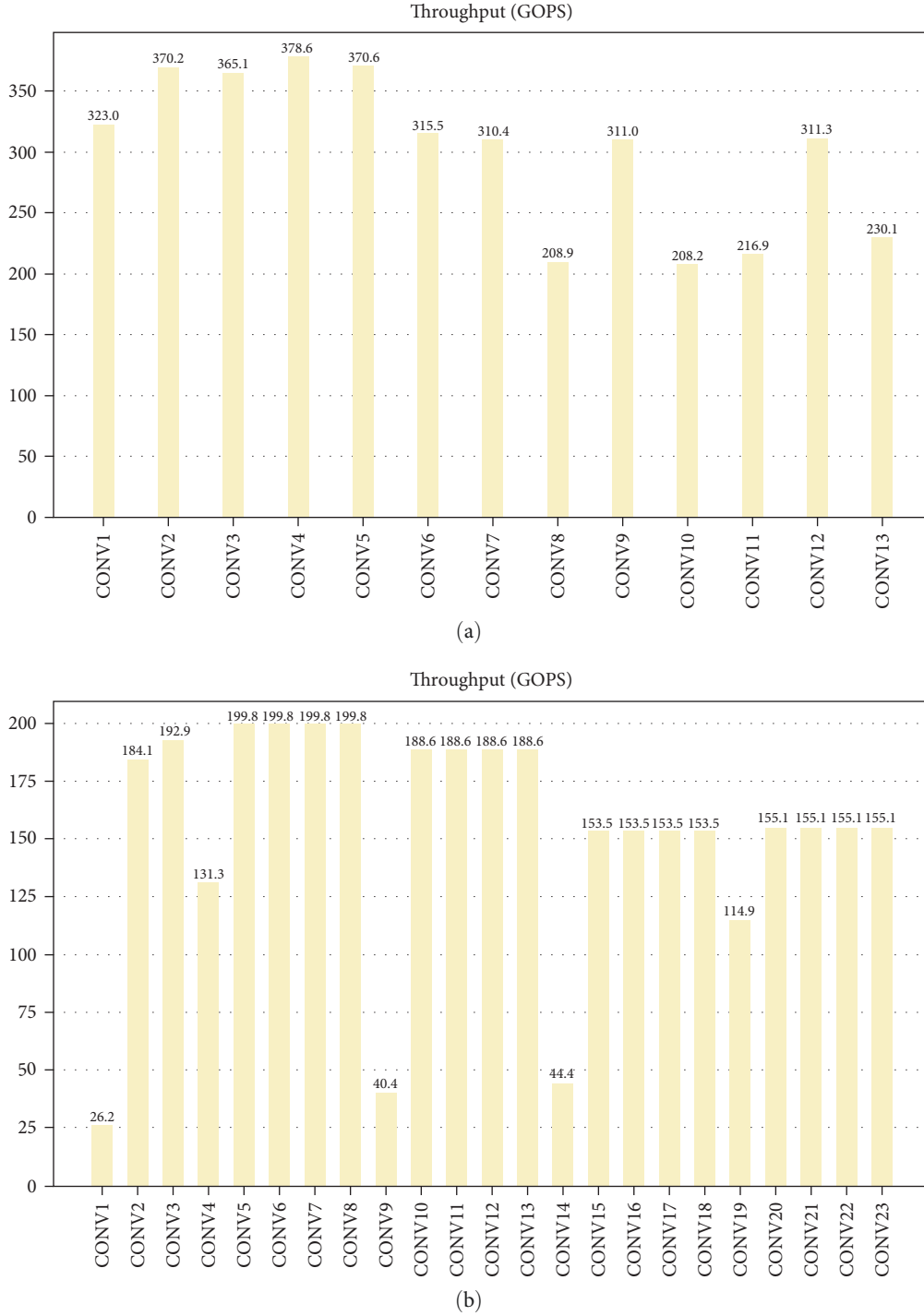


FIGURE 10: The throughput at each layer of the network: (a) YOLOv3-Tiny; (b) ResNet-18.

of predefined microinstruction with INT8 or INT16 quantization, respectively. A single array accelerator comprises  $16 \times 16$  PEs designs. Utilizing a single array design, the accelerator's average throughput reaches 153.14 GOPS for ResNet-18, with a power consumption of just 6.93 W. In the dual array configuration, the accelerator's throughput for YOLOv3-Tiny is 301.52 GOPS, while consuming only 10.68 W.

Table 7 summarizes the performance metrics of the proposed hardware accelerator, providing a comparison with

other FPGA accelerator works. Given the substantial variations among accelerators in terms of models, accuracy, resource utilization, and performance, this paper opts for a comparison based on DSP efficiency. The calculation method for the DSP efficiency is detailed in Equation (9).

$$\text{DSP efficiency} = \frac{\text{Throughput}}{\text{DSP utilization}}. \quad (9)$$

TABLE 6: Comparison with CPU and GPU.

|                    | CPU           | GPU          | Ours (single-core) | Ours (dual-core) |
|--------------------|---------------|--------------|--------------------|------------------|
| Platform           | Intel i7-7700 | GTX1080      | VC709              | VC709            |
| Technology (nm)    | 14            | 16           | 28                 | 28               |
| Precision          | 32-bit float  | 32-bit float | 16-bit fixed       | 8-bit fixed      |
| Speed (FPS)        | 3.9           | 88.7         | 51.0               | 102.0            |
| Power (W)          | 65.00         | 180.00       | 6.93               | 10.68            |
| Efficiency (FPS/W) | 0.06          | 0.49         | 7.36               | 9.55             |

TABLE 7: Comparison with related work.

|                           | [41]      | [42]        | [43]        | [44]        | Ours (single-core) | Ours (dual-core) |
|---------------------------|-----------|-------------|-------------|-------------|--------------------|------------------|
| Platform                  | XC7Z020   | XC7Z020     | XC7Z020     | XCK325T     | VX690T             | VX690T           |
| Network                   | ResNet-34 | MobileNetV2 | YOLOv2-Tiny | MobileNetV3 | ResNet-18          | YOLOv3-Tiny      |
| Precision                 | NA        | INT8        | INT16       | INT8        | INT16              | INT8             |
| Frequency (MHz)           | 100       | 200         | 200         | 200         | 200                | 200              |
| DSP utilization           | 392       | 392         | 224         | 704         | 514                | 1,028            |
| Power (W)                 | 6.511     | NA          | 5.348       | 8.530       | 6.931              | 10.680           |
| Throughput (GOPS)         | 57.02     | 96.40       | 57.24       | 84.48       | 153.14             | 301.52           |
| DSP efficiency            | 0.145     | 0.246       | 0.256       | 0.120       | 0.298              | 0.293            |
| Power efficiency (GOPS/W) | 8.76      | NA          | 10.70       | 9.90        | 22.09              | 28.23            |

In addition, the throughput-power ratio power efficiency is compared. Our DSP efficiency and power efficiency are better than other works. In [41], the frequency is only 100 MHz. In addition, its throughput and power efficiency are the worst. The DSP efficiency of [42, 43] is slightly lower than that of our study, but our throughput and power efficiency are much higher than them. In [44], it has high throughput while its DSP efficiency is the lowest because it uses more DSPs. In addition, our accelerator is more flexible and can be configured with different numbers of accelerator cores or quantization according to resource requirements, as well as other CNNs.

## 5. Conclusion

The main calculation of the CNN is the operation of convolution multiplication and accumulation. Based on the operation mode of the systolic array, this paper divides the convolution calculation into channel blocks and optimizes the calculation method to improve the utilization of on-chip resources and improve the calculation efficiency. A microinstruction encoding format for CNN is proposed, and an RTL-level CNN FPGA accelerator with microinstruction sequence scheduling data flow is designed. We establish YOLOv3-Tiny and ResNet-18 network models through the configuration of predefined microinstruction and use the dual-core structure with INT8 quantization and single-core structure with INT16 quantization to verify the two models, respectively. The experimental results show that our accelerator can achieve a performance of 301.52 GOPS on YOLOv3-Tiny while only consuming 10.68 W and a performance of 153.14 GOPS on ResNet-18 while only consuming 6.93 W. A notable drawback in this paper is the overuse of on-chip

BRAM resources. In future research, the system will undergo additional optimization through an increase in the main frequency of operation and resource reduction, aiming to enhance the utilization of computing resources and bandwidth.

## Data Availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

## Conflicts of Interest

The authors declare that they have no conflicts of interest.

## Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62171342 and the State Key Laboratory of Geo-Information Engineering, no. SKLGIE2023-M-3-1.

## References

- [1] S. Dong, P. Wang, and K. Abbas, "A survey on deep learning and its applications," *Computer Science Review*, vol. 40, Article ID 100379, 2021.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [3] C. Zuo, J. Qian, S. Feng et al., "Deep learning in optical metrology: a review," *Light: Science & Applications*, vol. 11, no. 1, p. 11, 2022.
- [4] T. Chen, Z. Du, N. Sun et al., "Diannao: a small-footprint high-throughput accelerator for ubiquitous machine-learning,"

- ACM SIGARCH Computer Architecture News, vol. 42, no. 1, pp. 269–284, 2014.
- [5] S. Liu, Z. Du, J. Tao et al., “Cambricon: an instruction set architecture for neural networks,” *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 393–405, 2016.
  - [6] S. Zhang, Z. Du, L. Zhang et al., “Cambricon-X: an accelerator for sparse neural networks,” in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–12, IEEE, October 2016.
  - [7] V. Gokhale, J. Jin, A. Dundar, B. Martini, and E. Culurciello, “A 240 G-ops/s mobile coprocessor for deep neural networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, IEEE, June 2014.
  - [8] X. Wei, C. H. Yu, P. Zhang et al., “Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs,” in *Proceedings of the 54th Annual Design Automation Conference 2017*, pp. 1–6, IEEE, June 2017.
  - [9] P. Ye, J. Huang, Z. Sun, M. Yang, and L. Meng, “The process and experience in the development of Chinese lunar probe,” *Science China-Technological Sciences*, vol. 44, no. 6, pp. 543–558, 2014.
  - [10] G. Yang, J. Lei, W. Xie et al., “Algorithm/hardware codesign for real-time on-satellite CNN-based ship detection in SAR imagery,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–18, 2022.
  - [11] J. Lin, X. Wu, Y. Chai, and H. Yin, “Structure optimization of convolutional neural networks: a survey,” *Acta Automatica Sinica*, vol. 46, no. 1, pp. 24–37, 2020.
  - [12] W. Songyan, M. Xiangqiang, and Y. Xiaobin, “Research on data acquisition technology of chinese high-resolution broadband multispectral satellites in the “the belt and road initiative” region,” *Remote Sensing Technology and Application*, vol. 38, no. 4, pp. 776–782, 2023.
  - [13] X. Wei, W. Liu, L. Chen, L. Ma, H. Chen, and Y. Zhuang, “FPGA-based hybrid-type implementation of quantized neural networks for remote sensing applications,” *Sensors*, vol. 19, no. 4, 2019.
  - [14] Y. Nong and J. Wang, “Real-time object detection in remote sensing images based on embedded system,” *Acta Optica Sinica*, vol. 41, no. 10, pp. 179–186, 2021.
  - [15] S. Wang, S. Zhang, X. Huang, and L. Chang, “Single-chip multi-processing architecture for spaceborne SAR imaging and intelligent processing,” *Journal of Northwestern Polytechnical University*, vol. 39, no. 3, pp. 510–520, 2021.
  - [16] S. Liu and W. Deng, “Very deep convolutional neural network based image classification using small training sample size,” in *2015 3rd IAPR Asian Conference on Pattern Recognition (ACPR)*, pp. 730–734, IEEE, Kuala Lumpur, Malaysia, November 2015.
  - [17] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, IEEE, Las Vegas, NV, USA, June 2016.
  - [18] F. Chollet, “Xception: deep learning with depthwise separable convolutions,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1800–1807, IEEE, July 2017.
  - [19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: inverted residuals and linear bottlenecks,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, IEEE, June 2018.
  - [20] A. Howard, M. Sandler, B. Chen et al., “Searching for mobilenetV3,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, IEEE, Seoul, Korea (South), October 2019.
  - [21] J. Redmon and A. Farhadi, “YOLO9000: better, faster, stronger,” in *Proceedings of the IEEE Conference On Computer Vision and Pattern Recognition*, pp. 7263–7271, IEEE, July 2017.
  - [22] Z. Liquan and L. Shuaiyang, “Object detection algorithm based on improved YOLOv3,” *Electronics*, vol. 9, no. 3, 2020.
  - [23] X. Hu, Y. Liu, Z. Zhao et al., “Real-time detection of uneaten feed pellets in underwater images for aquaculture using an improved YOLO-V4 network,” *Computers and Electronics in Agriculture*, vol. 185, Article ID 106135, 2021.
  - [24] P. Adarsh, P. Rathi, and M. Kumar, “YOLO v3-Tiny: object detection and recognition using one stage improved model,” in *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pp. 687–694, IEEE, March 2020.
  - [25] Y. Cai, C. Liang, Z. Tang, H. Li, and S. Gong, “Deep neural network with limited numerical precision,” in *International Conference on Applications and Techniques in Cyber Security and Intelligence*, J. Abawajy, K.-K. R. Choo, and R. Islam, Eds., pp. 42–50, Springer International Publishing, Cham, 2018.
  - [26] A. Boutros, M. Eldafrawy, S. Yazdanshenas, and V. Betz, “Math doesn’t have to be hard: logic block architectures to enhance low-precision multiply-accumulate on FPGAs,” in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA ’19*, pp. 94–103, Association for Computing Machinery, New York, NY, USA, 2019.
  - [27] J. Sun and X. Fang, “Mixed-precision quantization technology of convolutional neural networks,” *Information Technology*, vol. 44, no. 6, Article ID 4, 2020.
  - [28] H. Zeng, X. Lin hao Hu, J. Hou, and C. Cai, “Deep neural network compression and acceleration: an overview,” *Journal of Signal Processing*, vol. 38, no. 1, Article ID 12, 2022.
  - [29] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer, “ZeroQ: a novel zero shot quantization framework,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13166–13175, IEEE, 2020.
  - [30] Y. Xu, D. Zhang, L. Wang, and B. Hua, “Lightweight feature fusion network design for local feature recognition of non-cooperative target,” *Infrared and Laser Engineering*, vol. 49, no. 7, Article ID 20200170, 2020.
  - [31] L. Lv, *Research on neural network compression for SSDResNet*, Ph.D. thesis, XIDIAN University, Xian, 2019.
  - [32] B. Jacob, S. Kligys, B. Chen et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, IEEE, June 2018.
  - [33] R. Gong, X. Liu, S. Jiang et al., “Differentiable soft quantization: bridging full-precision and low-bit neural networks,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4851–4860, IEEE, October 2019.
  - [34] F. Zhu, R. Gong, F. Yu et al., “Towards unified INT8 training for convolutional neural network,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, June 2020.
  - [35] Y. Ma, N. Suda, Y. Cao, J. S. Seo, and S. Vrudhula, “Scalable and modularized RTL compilation of convolutional neural networks onto FPGA,” in *International Conference on Field Programmable Logic Applications*, pp. 1–8, IEEE, 2016.
  - [36] X. Wei, C. H. Yu, P. Zhang et al., “Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs,” in *2017 54th ACM/EDAC/IEEE Design Automation Conference*, pp. 1–6, IEEE, June 2017.
  - [37] Y. Shen, M. Ferdman, and P. Milder, “Maximizing CNN accelerator efficiency through resource partitioning,” *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 535–547, 2017.
  - [38] C. Zhang, W. Di, J. Sun, G. Sun, G. Luo, and J. Cong, “Energy-efficient CNN implementation on a deeply pipelined FPGA

- cluster,” in *Proceedings of the 2016 International Symposium on Low Power Electronics and Design*, pp. 326–331, Association for Computing Machinery, 2016.
- [39] D. Wang, J. Shen, M. Wen, and C. Zhang, “An efficient design flow for accelerating complicated-connected CNNs on a multi-FPGA platform,” in *Proceedings of the 48th International Conference on Parallel Processing*, pp. 1–10, Association for Computing Machinery, 2019.
- [40] N. P. Jouppi, D. H. Yoon, G. Kurian et al., “A domain-specific supercomputer for training deep neural networks,” *Communications of the ACM*, vol. 63, no. 7, pp. 67–78, 2020.
- [41] X. Hu, T. Chen, H. Huang, Z. Liu, X. Li, and X. Xiong, “Efficient field-programmable gate array-based reconfigurable accelerator for deep convolution neural network,” *Electronics Letters*, vol. 57, no. 6, pp. 238–240, 2021.
- [42] X. Li, H. Huang, Y. Liu, X. Hu, and X. Xiong, “A digital signal processor-efficient accelerator for depthwise separable convolution,” *Electronics Letters*, vol. 58, no. 7, pp. 271–273, 2022.
- [43] H. Huang, X. Hu, X. Li, and X. Xiong, “An efficient loop tiling framework for convolutional neural network inference accelerators,” *IET Circuits, Devices & Systems*, vol. 16, no. 1, pp. 116–123, 2022.
- [44] Y. Yu, T. Zhao, K. Wang, and L. He, *Light-OPU: An FPGA-based Overlay Processor for Lightweight Convolutional Neural Networks*, Association for Computing Machinery, New York, NY, USA, 2020.